

1. Task Description

In this assignment, you will design a 5-staged pipelined RISC-V processor in SystemVerilog. The processor must operate in-order and single-issue.

Required instruction set: **RV32I custom instruction set in the HW1**

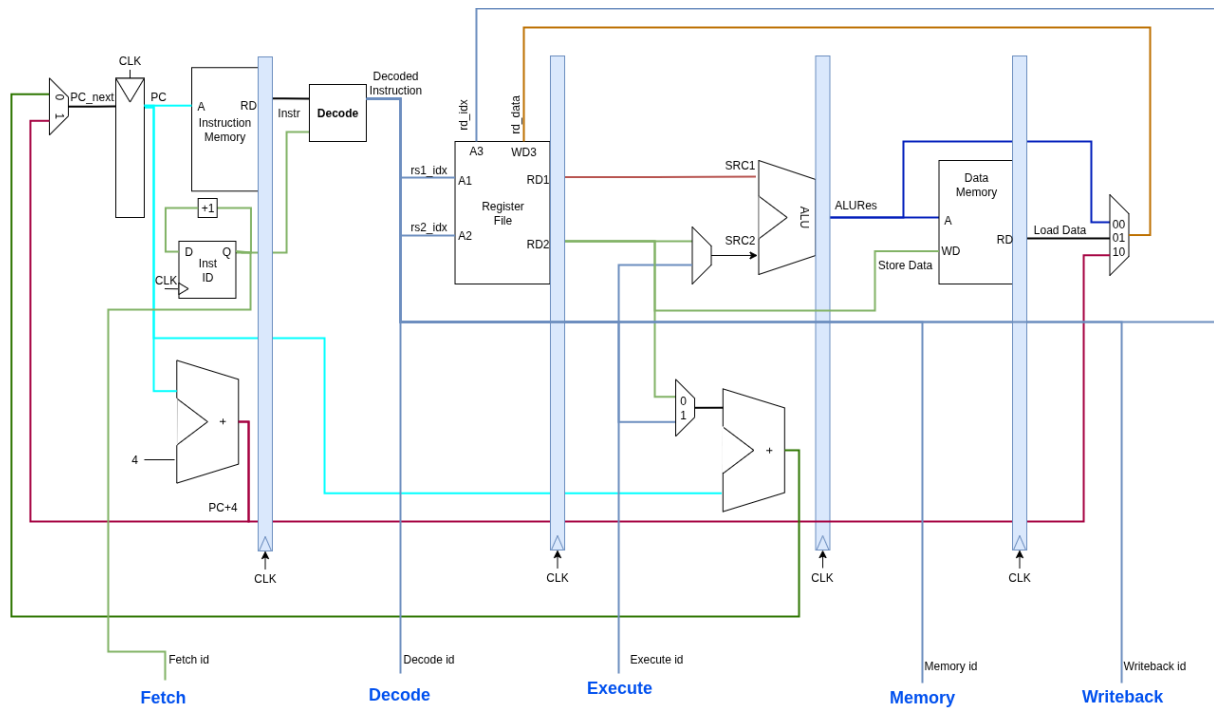
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
imm[31:20]												1110101				imm[19:12]						rd				LUI						
imm[31:20]												1110100				imm[19:12]						rd				AUIPC						
imm[20 10:1 11]												1101111				imm[19:12]						rd				JAL						
imm[11:0]												1100111				rs1			000			rd				JALR						
imm[12 10:5]						rs2						1100011				rs1			100			imm[4:1 11]				BEQ						
imm[12 10:5]						rs2						1100011				rs1			101			imm[4:1 11]				BNE						
imm[12 10:5]						rs2						1100011				rs1			000			imm[4:1 11]				BGEU						
imm[12 10:5]						rs2						1100011				rs1			001			imm[4:1 11]				BLTU						
imm[12 10:5]						rs2						1100011				rs1			010			imm[4:1 11]				BGE						
imm[12 10:5]						rs2						1100011				rs1			011			imm[4:1 11]				BLT						
imm[11:0]												1100000				rs1			000			rd				LBU						
imm[11:0]												1100000				rs1			001			rd				LHU						
imm[11:0]												1100000				rs1			100			rd				LB						
imm[11:0]												1100000				rs1			101			rd				LH						
imm[11:0]												1100000				rs1			110			rd				LW						
imm[11:5]						rs2						1100001				rs1			000			imm[4:0]				SB						
imm[11:5]						rs2						1100001				rs1			001			imm[4:0]				SH						
imm[11:5]						rs2						1100001				rs1			010			imm[4:0]				SW						
imm[11:0]												1100100				rs1			000			rd				ADDI						
imm[11:0]												1100100				rs1			010			rd				SLTI						
imm[11:0]												1100100				rs1			011			rd				SLTIU						
imm[11:0]												1100100				rs1			100			rd				ORI						
imm[11:0]												1100100				rs1			110			rd				XORI						
imm[11:0]												1100100				rs1			111			rd				ANDI						
0000000						imm[4:0]						1100100				rs1			001			rd				SLLI						
0000000						imm[4:0]						1100100				rs1			101			rd				SRLI						
0000010						imm[4:0]						1100100				rs1			101			rd				SRAI						
0000000						rs2						1110001				rs1			000			rd				ADD						
0000010						rs2						1110001				rs1			000			rd				SUB						
0000000						rs2						1110001				rs1			001			rd				SLL						
0000000						rs2						1110001				rs1			010			rd				SLT						
0000000						rs2						1110001				rs1			011			rd				SLTU						
0000000						rs2						1110001				rs1			100			rd				XOR						
0000000						rs2						1110001				rs1			101			rd				SRL						
0000010						rs2						1110001				rs1			101			rd				SRA						
0000000						rs2						1110001				rs1			110			rd				OR						
0000000						rs2						1110001				rs1			111			rd				AND						

Note: This homework is mostly same with the previous one. The difference is being multicycle and pipelined architecture. Some extra control signals added for forcing pipelined architecture.

2. Expected Architecture

You are expected to create a 5-stage pipelined processor. A guiding illustration of the expected processor is given in figure below. Note that not all signals are given in the figure. You are expected to handle all data hazards by implementing stall and flush signals appropriately. Also, it is allowed to change the design as long as 5-staged pipeline is preserved.

You are also expected to assign an id value for each instruction and give them as output accordingly. For this purpose, an “id” and “pc” signal is added to dinstr_t type. The id of “0” will be ignored and the first instruction should start from id value of “1”. You can increment id values as you like as long as it keeps increasing for each instruction.



3. Top Module Interface

The top-level module of your design must have the following interface. Do not change the port names or directions.

```
module riscv_multicycle
    import riscv_pkg::*;
    #(
        parameter DMemInitFile = "dmem.mem",
        parameter IMemInitFile = "imem.mem"
    ) (
        input logic          clk_i,          // system clock
        input logic          rstn_i,         // system reset
        input logic [XLEN-1:0] addr_i,       // memory address input for reading
        output logic         update_o,       // retire signal
        output logic [XLEN-1:0] data_o,      // memory data output for reading
        output logic [XLEN-1:0] pc_o,        // retired program counter
        output logic [XLEN-1:0] instr_o,     // retired instruction
        output logic [4:0] reg_addr_o,       // retired register address
        output logic [XLEN-1:0] reg_data_o,  // retired register data
        output logic [XLEN-1:0] mem_addr_o,  // retired memory address
        output logic [XLEN-1:0] mem_data_o,  // retired memory data
        output logic         mem_wrt_o,     // retired memory write enable
        output logic [XLEN-1:0] fetch_id_o  // id of instruction on fetch stage
        output logic         fetch_valid_o  // valid instruction on fetch stage
        output logic [XLEN-1:0] decode_id_o // id of instruction on decode stage
        output logic         decode_valid_o // valid instruction on decode stage
        output logic [XLEN-1:0] execute_id_o // id of instruction on execute stage
        output logic         execute_valid_o // valid instruction on execute stage
        output logic [XLEN-1:0] mem_id_o    // id of instruction on mem stage
        output logic         mem_valid_o    // valid instruction on mem stage
        output logic [XLEN-1:0] writeback_id_o // id of instruction on writeback stage
        output logic         writeback_valid_o // valid instruction on writeback stage
    );
endmodule
```

4. Memory Initialization

The instruction and data memories of the processor must be initialized from files specified via the IMemInitFile and DMemInitFile parameters respectively. Both memories must be loaded using the \$readmemh .

```
$readmemh(IMemInitFile, instr_mem, 0, 2047);
$readmemh(DMemInitFile, data_mem, 0, 2047);
```

5. Repository Structure

The assignment repository has the following structure:

src/	Only modify files in this directory
tb/	Testbench (do not modify)
tests/	Test cases (.hex and .log files)
imem.mem	Instruction memory initialization
file dmem.mem	Data memory initialization file
Makefile	Build and test automation.
check_result.py	Automated result checker

6. Required Tools

The following tool must be installed to generate the coverage report: (Detailed expression of coverage report given in section 9.)

```
| sudo apt install lcov
```

In order to visualize behaviour of your processor you can use the following tool. You can follow the instructions given in github page or you can use pre-built binaries given in repository. Usage of this program is not mandatory but it will help you better understand the topic and may be much more helpful on the upcoming homeworks.

```
| https://github.com/shioyadan/Konata
```

7. Test Flow

Each test case is located in a separate subdirectory under tests/. Each subdirectory contains a .hex file (instruction memory contents) and a .log file (expected output).

Available tests:

- tests/base_test/
- tests/book_test/

Step 1 - Select a test.

Set the TK_TEST environment variable to the name of the test directory you want to use:

```
| export TK_TEST=base_test
```

Step 2 - Load the instruction memory.

Copy the selected test's imem.mem file into the root directory's imem.mem. For example, for BASE_TEST, copy "tests/base_test/imem.mem" into the root imem.mem.

Note: This step can be done manually. The processor reads imem.mem via \$readmemh at simulation startup, so the file must contain the correct contents before simulation begins.

Note: \$ make all FLAG=<test_directory_name> (example: \$ make all FLAG=base_test) will automatically copy tests/<test_directory_name>/imem.mem into the root imem.mem and \$ make run will check results.

Step 3 - Compare results.

```
| make run  
| make check
```

This command compares model.log against the golden log of the selected test. Results are reported per instruction category (R-type, I-ALU, Load, Store, Branch, Jump, etc.) as pass or fail.

8. Verifying Your Output

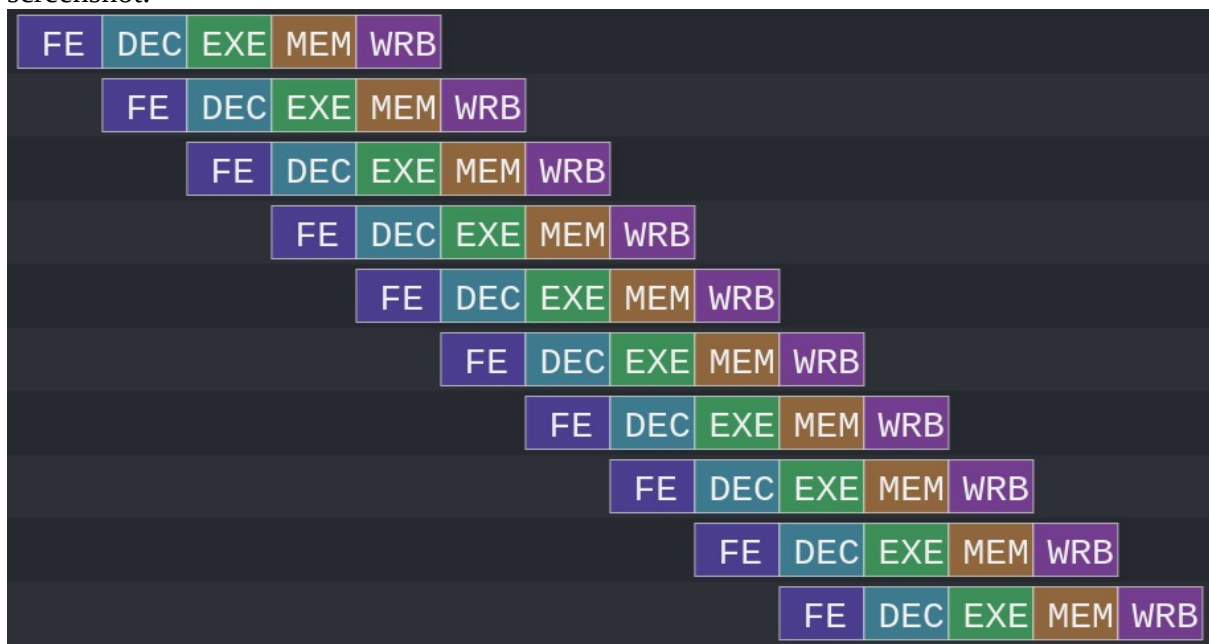
The `make check` command and the `check_result.py` script are provided solely as a convenience tool to help you identify differences between your output and the expected output. They are not part of the grading criteria themselves.

Your primary responsibility is to ensure that your `model.log` matches the golden log file (`tests/<TK_TEST>/test.log`) for the selected test. You are encouraged to compare these two files directly using a text comparison tool. Both VSCode and Notepad++ offer built-in file compare features that make it straightforward to spot differences line by line.

In addition, newly added output signals checks for verifying pipelined architecture. These checks are made in runtime and stops the simulation if an unexpected behaviour occurs. In order to bypass these checks during development, you can send ``valid`` and ``id`` values as ``0``, or you can use ``make run_without_pipeline``. However, in the final submission you have to make sure your design indeed works pipelined and, of course, correctly.

9. Visualizing Your Pipeline (Bonus, 5 points)

You can use Konata tool to visualize how your processor behaves. When you run your simulation, using signals added for this project, a file named ``konata.log`` is created on your working directory. When this log file is opened with the tool -and if your implementation is correct- it shows you how your processor behaves. The sample output is given in following screenshot:



Please note that the log file is created by your signals and the logs are created by an expected behaviour. Your implementation might include some nuances that are correct but still does not produce expected visualization in tool.

In order to get 5 point bonus, you have to take a screenshot of your pipeline visualization including at least 10 instructions and put it in your base homework folder with the name `"pipeline.png"`.

10. Writing Your Own Tests (Optional)

If you would like to write your own basic RISC-V assembly programs to test your processor, you do not need to install a toolchain. An online RISC-V assembler is available at:

| <https://riscvasm.lucasteske.dev/>

You can write RV32I assembly directly in the browser and obtain the corresponding machine code, which can then be used as the contents of `imem.mem` with help of `mtb_hex2hex_file.sh` file in the tests folder. You can write custom C code for testing too. However, you do not have any `golden.log` file for your custom tests in the tests folder.

11. Coverage Report

The coverage report measures how much of your design is exercised by the tests. It tracks which lines of code and which branches (if/else, case arms) were actually executed during simulation. Lines that were never reached are highlighted, giving you a clear picture of which parts of your design have not been tested yet.

To generate the report:

| `make cov`

The report is generated under `logs/html/`. Open `logs/html/index.html` in your browser to view the results. You can drill down into individual source files to see exactly which lines were covered.

Important: Coverage results depend entirely on which test you run. For example, `BASE_TEST` does not contain any load or store instructions, so the load/store related branches in your design will appear as uncovered. This does not mean your implementation is wrong — it simply means those paths were not exercised by that particular test. To get higher coverage, run tests that include a wider variety of instructions.

12. A Note on AI Tools

You are free to use AI assistants as a learning aid, but we strongly encourage you to avoid asking them to write your design from scratch. Processor design is a hands-on skill — the real learning happens when you struggle with the details, make mistakes, and figure out why something does not work. A design that passes the tests but that you do not fully understand will leave you unprepared for more advanced topics. Use AI to clarify concepts, check your reasoning, or get unstuck — not to skip the process entirely.

13. Important Notes

- The `dmem.mem` file must be left empty. Do not write any content into it.
- Do not add any `-Wno` flags to the Makefile. The design must compile without suppressing warnings.
- In case of a grading dispute, you will be required to download the exact file you submitted from Ninova, record a screen capture demonstrating the system running correctly, upload the video to YouTube as an unlisted video, and share the link.
- **Don't send your code before testing.**

14. Grading Criteria

Criteria	Grades
linter	10p
build	5p
run	5p
base_test	10p
Code Quality	15p
Dmem/Imem design	5p
Custom Tests	50p
Pipeline Visualization	5p